

Binary Search — Study Guide

From "find the target" to "find the boundary" to "search the answer space"

Derived from a Socratic ChatGPT lesson (249-page transcript, Sept 2026)
Every code sample is pulled live from a tested `sol.py` and verified against brute force.

Roadmap

This guide follows one mental model end to end:

Binary search repeatedly examines one position and uses an ordering or monotonic property to eliminate roughly half of the remaining search space.

Everything else — lower bound, Koko's bananas, rotated arrays, peak finding, 2D matrices — is a variation of that one idea applied to a different search space. The guide is built in the same order the source lesson taught it:

1. **Fundamentals** — search space, invariant, why `left <= right` vs `left < right`
2. **Exact search & boundaries** — Pattern 1 (exact), lower/upper bound, First/Last True
3. **Binary search on the answer** — the mental leap: search *values*, not indices
4. **Minimum feasible answer problems** — Koko, Shipping, Smallest Divisor, Bouquets, Split Array
5. **Maximum feasible answer problems** — Aggressive Cows / Magnetic Force, Closest Target Sum
6. **Square roots** — integer (Last True) and floating-point (interval shrink)
7. **Rotated sorted arrays** — exploiting "at least one half is sorted"
8. **Peak / local structure** — binary search without a sorted array
9. **Matrix search** — globally sorted vs. row+col sorted (staircase)
10. **More boundary problems** — Search Insert, First/Last Position, First Bad Version, Find the Duplicate, and others
11. **Median of Two Sorted Arrays** — the hardest classic, via partitions
12. **Recognition & pitfalls** — binary search vs. two pointers/sliding window/hashmap, common mistakes, a real correction the model had to make mid-lesson
13. **Templates & decision guide** — the four skeletons worth memorizing
14. **Recognition drills** — practice identifying the pattern before solving
15. **Interview Q&A** — rapid-fire questions an interviewer might actually ask

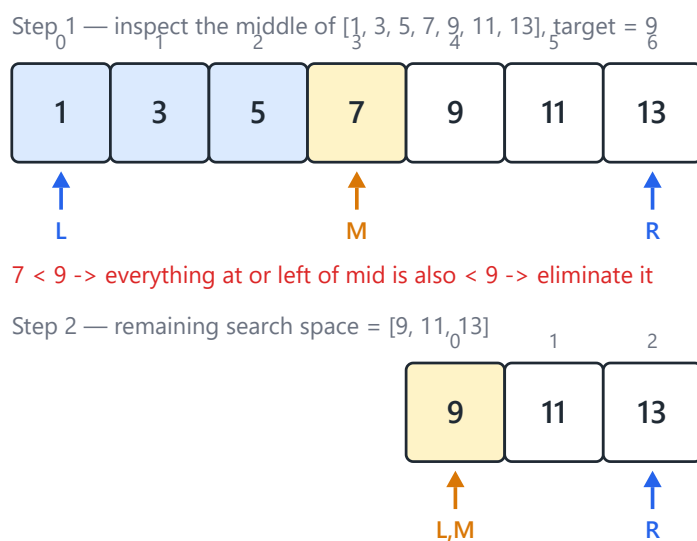
How to use this guide

Every pattern follows: **Understand** → **Recognize** → **Derive** → **Solve** → **Implement** → **Recall**, not "memorize template → tweak conditions." Each code block is the *exact* tested source from `sol.py` (see `test_sol.py` : every function is checked against hundreds of random trials, several against Python's own `bisect` module or a brute-force $O(n)$ reference). Each dry run in this guide is produced by an instrumented copy of the algorithm in `traces.py` and **asserted** to agree with `sol.py` 's real answer before the guide is built — a dry run can never silently drift out of sync with the code.

Part 1 — Fundamentals

1.1 The core idea

Forget code. Take `[1, 3, 5, 7, 9, 11, 13]` and look for `9`. A linear scan would inspect `1` → `3` → `5` → `7` → `9`, up to five comparisons. Binary search instead inspects the *middle* element and asks what that one comparison teaches about the **entire rest of the array**.



`7 < 9`, and because the array is sorted, *everything to the left of 7 is also less than 9*. Not "probably" — provably. That single comparison lets us throw away four elements without looking at them. That is the whole trick:

Inspect one position → prove something about a whole region → eliminate that region → repeat.

The reason this is $O(\log n)$: each step keeps roughly half of what came before, so after k steps the remaining space is $n / 2^k$. Setting that to 1 gives $k \approx \log_2 n$.

1.2 Search space: indices, not values

It's tempting to think "we're searching through the values." We aren't. Look at what the function actually returns:

```
return mid # an index, not a value
```

What binary search actually narrows down: indices, not values



indices -> the search space we eliminate from



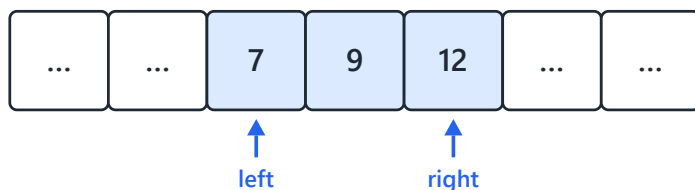
values -> only used to DECIDE which half to eliminate

The **values** (`nums[mid]`) are only ever used to *decide* which indices to keep. The **search space being narrowed down is** `[left, right]` , a **range of indices**. This distinction matters enormously once we get to binary search on the answer (Part 3), where there is no array at all — the search space becomes a range of *possible answer values*, and `mid` becomes a candidate answer rather than a candidate index.

1.3 The invariant

Every correct binary search maintains an invariant — a statement that stays true after every single update. For ordinary search over a sorted array:

Invariant: if the target exists, it is always inside `[left, right]`



Every update must preserve this statement — that's how you debug binary search.

If the target exists, it is always somewhere inside `[left, right]`.

This is the best debugging tool binary search has. Whenever you write a boundary update, ask: "Could this update have thrown away the answer?" If yes, the logic is wrong. Two concrete consequences:

- `nums[mid] < target` $\Rightarrow$ `mid` **and everything before it** is provably too small $\Rightarrow$ safe to set `left = mid + 1` (`mid` is *eliminated*, not just skipped).
- `nums[mid] > target` $\Rightarrow$ `right = mid - 1`, symmetric reasoning.

1.4 `left <= right` VS. `left < right`

This single choice trips up more binary-search code than any other detail. Stop trying to memorize "which loop for which problem" and ask instead:

What happens when exactly one candidate remains — do I still need to check it, or does my invariant already tell me the answer?

Which loop condition?

while `left <= right`

"Do I still need to check the one remaining candidate?"

`mid` gets ELIMINATED
(exact-value search)

while `left < right`

"Am I narrowing until one candidate remains?"

`mid` may be KEPT
(boundary / answer search)

- **Exact-value search** (Pattern 1 below): when one candidate index is left, we still don't know if `nums[that index] == target`. We must check it. Loop while `left <= right`; the branches *eliminate* `mid` explicitly (`left = mid + 1` / `right = mid - 1`).
- **Boundary / answer search** (everything from Part 2 onward): we are *narrowing toward* one candidate, and the invariant we maintain ("the answer lies in `[left, right]`") already guarantees that the last remaining position **is** the answer. Loop while `left < right`; one branch may *keep* `mid` (`left = mid` or `right = mid`) because `mid` itself could still be the answer.

Every pattern in this guide reduces to one of those two loop shapes. Getting this choice right, and pairing it with the correct midpoint (below), is 90% of writing correct binary search from scratch.

1.5 Overflow-safe midpoint

```
mid = (left + right) // 2      # can overflow in fixed-width languages
mid = left + (right - left) // 2  # equivalent, but never overflows
```

Python integers are arbitrary precision, so this never matters for correctness here — but it's the form worth building as a habit, because the same code translated to Java/C++ would silently misbehave on huge ranges without it. Every function in `sol.py` uses the safe form.

Part 2 — Exact Search, Boundaries, and the Two Universal Templates

2.1 Pattern 1 — Exact Binary Search

Recognition signal: sorted array + "does this exact value exist, and where?"

Search space: indices `[0, n-1]`. At `mid`: `nums[mid] == target` → found; `nums[mid] < target` → discard `[left, mid]`; `nums[mid] > target` → discard `[mid, right]`. Loop `while left <= right` (Section 1.4) because the last candidate still needs checking.

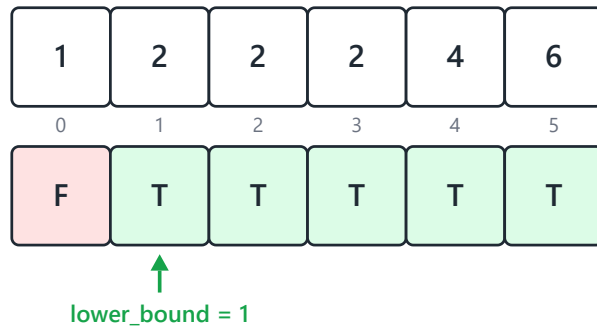
```
def binary_search(nums: Sequence[int], target: int) -> int:
    """Pattern 1 -- exact target search. Returns index or -1."""
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space.

2.2 Pattern 2 — Lower Bound (First True)

Now introduce duplicates: `[1, 2, 2, 2, 4, 6]`, and ask a different question: "*find the first position where* `nums[i] >= 2`." Turn it into a predicate and evaluate it conceptually at every index:

nums = [1, 2, 2, 2, 4, 6], target = 2 -> predicate: nums[i] >= 2



The predicate `nums[i] >= target` is **monotonic** — once true, it stays true. That monotonicity is the *only* thing binary search actually needs; the array doesn't have to be sorted by *equality*, just by this predicate. We are now searching for **the first True**, and the key realization is:

We never materialize the Boolean array. We evaluate `condition(mid)` in $O(1)$ (or whatever the check costs) and let monotonicity tell us about the whole region. Building `[x >= 2 for x in nums]` up front would be $O(n)$ and defeats the purpose.

At `mid`: if `condition(mid)` is **False**, `mid` and everything before it are False too → `left = mid + 1` (mid eliminated). If `condition(mid)` is **True**, `mid` *could* be the first True → we must keep it → `right = mid`. Because a branch keeps `mid`, this is boundary-search style: loop `while left < right`, lower midpoint.

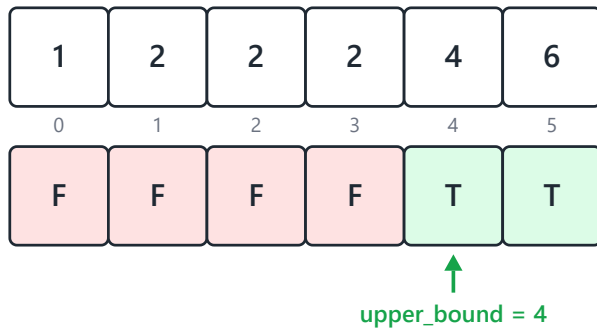
```
def lower_bound(nums: Sequence[int], target: int) -> int:
    """First index i with nums[i] >= target (== len(nums) if none)."""
    left, right = 0, len(nums)
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] >= target:
            right = mid
        else:
            left = mid + 1
    return left
```

Two edge-case details worth noting: `right` starts at `len(nums)`, not `len(nums) - 1` — the lower bound can legitimately equal `n` when every element is smaller than the target (nothing to insert before). And this is *exactly* `first_true(0, len(nums), lambda i: nums[i] >= target)`.

2.3 Pattern 3 — Upper Bound (also First True)

Same idea, different predicate: first index where `nums[i] > target`.

`nums = [1, 2, 2, 2, 4, 6], target = 2` -> predicate: `nums[i] > 2`



```
def upper_bound(nums: Sequence[int], target: int) -> int:
    """First index i with nums[i] > target (== len(nums) if none)."""
    left, right = 0, len(nums)
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] > target:
            right = mid
        else:
            left = mid + 1
    return left
```

The relationship that makes both useful together:

Quantity	Formula
First occurrence of <code>target</code>	<code>lower_bound(target)</code>
Last occurrence of <code>target</code>	<code>upper_bound(target) - 1</code>
Count of <code>target</code>	<code>upper_bound(target) - lower_bound(target)</code>
Insertion index for a value not present	<code>lower_bound(target)</code>

Mnemonic: **LB = first `>=`**. **UB = first `>`**. Both are the exact same First True template with a one-character difference in the condition.

2.4 The two universal templates

Every boundary problem in this guide — lower bound, upper bound, "minimum X such that...", "maximum X such that..." — is one of exactly two shapes. Internalizing these two is worth more than memorizing twenty problem-specific solutions.

First True (find the smallest x where a monotonic predicate turns True)

First True — find the boundary where a monotonic predicate turns True



↑
first True

condition(mid) False -> left = mid+1 | True -> right = mid (keep mid)

```
def first_true(low: int, high: int, condition: Callable[[int], bool]) -> int:
    """Smallest x in [low, high] with condition(x) True. condition must be
    monotonic: False...False True...True. Assumes condition(high) is True."""
    left, right = low, high
    while left < right:
        mid = left + (right - left) // 2
        if condition(mid):
            right = mid
        else:
            left = mid + 1
    return left
```

Last True (find the largest x where a monotonic predicate is still True)

Last True — find the boundary where a monotonic predicate turns False



↑
last True

condition(mid) True -> left = mid (keep mid) | False -> right = mid-1

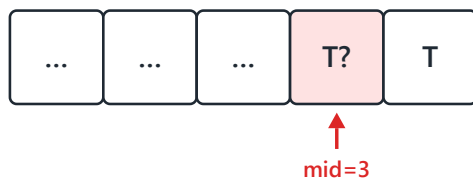
```
def last_true(low: int, high: int, condition: Callable[[int], bool]) -> int:
    """Largest x in [low, high] with condition(x) True. condition must be
    monotonic: True...True False...False. Assumes condition(low) is True."""
    left, right = low, high
    while left < right:
        mid = left + (right - left + 1) // 2
        if condition(mid):
            left = mid
        else:
            right = mid - 1
    return left
```

2.5 Why Last True needs the *upper* midpoint

This is the single most common source of infinite loops in hand-written binary search. Suppose `left = 3`, `right = 4` and we're finding Last True. The ordinary (lower) midpoint gives `mid = (3+4)//2 = 3`. If `condition(3)` is True, the update is `left = mid`, i.e. `left = 3` — **nothing changed**, and the loop spins forever.

Why Last True needs the UPPER midpoint: `left=3`, `right=4`

Lower mid: $mid = (3+4)/2 = 3$. If `condition(3)` is True -> `left = mid = 3`. Nothing moved.



Upper mid: $mid = (3+4+1)/2 = 4$. If `condition(4)` is True -> `left = mid = 4`. Progress.



The fix: whenever a branch does `left = mid` (i.e. *keeps* `mid` while moving `left` forward), bias the midpoint up: `mid = left + (right - left + 1) // 2`. This guarantees `mid > left` whenever `left < right`, so `left = mid` always makes progress. Symmetrically, First True's `right = mid` branch is always safe with the ordinary (lower) midpoint, because `mid < right` whenever `left < right`.

The rule, stated once and for all:

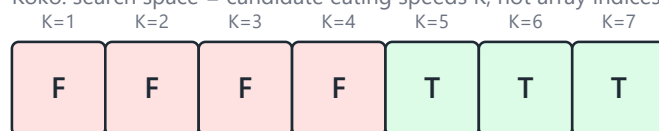
I keep <code>mid</code> on...	I therefore use...
the True branch, moving <code>right</code> down to <code>mid</code> (First True)	lower midpoint <code>(left+right)//2</code>
the True branch, moving <code>left</code> up to <code>mid</code> (Last True)	upper midpoint <code>(left+right+1)//2</code>

Part 3 — Binary Search on the Answer

3.1 The mental leap

Everything so far searched indices into an array. Now drop the array entirely. Consider: "Koko has piles of bananas and eats at a constant speed K bananas/hour. Find the minimum K that lets her finish within H hours." There is no sorted array to probe. But there is still a **range of possible answers**: $K = 1, 2, 3, \dots$. For any candidate K we can ask a yes/no question — "can she finish in time at speed K ?" — and that question is **monotonic**: if speed 5 works, every speed ≥ 5 also works (eating faster never makes you slower).

Koko: search space = candidate eating speeds K , not array indices



↑
min feasible K

feasible(K) = hours needed at speed $K \leq H$ (checked in $O(n)$, never materialised as an array)

That's the entire recognition test for this pattern:

1. **Search space** — the range of possible answer values, not indices.
2. **mid** — a *candidate answer*, not a candidate index.
3. **condition(mid)** — "is this candidate feasible?", answered by a problem-specific check (often $O(n)$).
4. **Monotonicity** — feasibility only moves in one direction as the candidate increases.

If all four hold, this is **binary search on the answer**, and it reduces immediately to whichever universal template (Part 2.4) matches the question:

Question shape	Template
" Minimum X such that X works"	First True
" Maximum X such that X works"	Last True

3.2 Two-layer complexity

Binary search itself only costs $O(\log(\text{range}))$ iterations — but each iteration pays for a feasibility check that is usually $O(n)$. The total is therefore $O(n \log(\text{range}))$, **not** $O(\log n)$. Don't let the phrase "binary search" imply cheap by default — the feasibility function is where the real work (and the real complexity) usually lives.

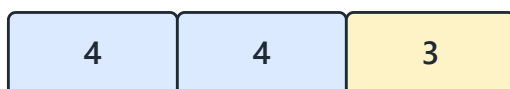
```
Binary Search
  |
candidate answer `mid`
  |
Feasibility check  <- usually the expensive O(n) part
  |
True / False
  |
eliminate half the answer range
```

3.3 The ceiling-division trick

Several feasibility checks below need $\text{ceil}(a / b)$ using only integer division (no floats, no rounding surprises):

Ceiling division without floats: $\text{ceil}(p / k) == (p + k - 1) // k$

$p = 11, k = 4 \rightarrow (11 + 4 - 1) // 4 = 14 // 4 = 3$



3 whole hours needed to eat all 11 bananas at speed 4/hr (last hour partial)

```
ceil(a / b) == (a + b - 1) // b    for positive integers a, b
```

Why it works: if a divides evenly, adding $b - 1$ still lands the division on the same quotient ($(a + b - 1) // b == a // b$). If there's a remainder, adding $b - 1$ is guaranteed to push the numerator past the next multiple of b , bumping the quotient up by exactly one. You'll reach for this constantly — batches, pages, groups, hours, containers.

Part 4 — Minimum Feasible Answer (First True over a value range)

Every problem here reduces to: *pick the search bounds*, write `feasible(x)`, call `first_true`. The skill is entirely in identifying the bounds and the feasibility check — the binary search around it never changes.

4.1 LC 875 — Koko Eating Bananas

`piles = [3, 6, 7, 11]`, `h = 8`. Minimum eating speed `k` so all piles finish within `h` hours.

- **Search space:** `k ∈ [1, max(piles)]` — slower than 1 makes no sense; eating an entire largest pile in one hour is always enough.
- `feasible(K)`: `sum(ceil(pile / K) for pile in piles) <= h`.
- **Monotonic** because increasing `k` can only reduce hours needed.

```
def min_eating_speed(piles: Sequence[int], h: int) -> int:
    def feasible(k: int) -> bool:
        return sum((p + k - 1) // k for p in piles) <= h

    return first_true(1, max(piles), feasible)
```

Dry run (`piles=[3,6,7,11]`, `h=8`):

left	right	mid	hours(mid)	action
1	11	6	6	hours(6)=6 <= 8? True -> right = mid
1	6	3	10	hours(3)=10 <= 8? False -> left = mid+1
4	6	5	8	hours(5)=8 <= 8? True -> right = mid
4	5	4	8	hours(4)=8 <= 8? True -> right = mid

answer: min speed 4

4.2 LC 1011 — Capacity To Ship Packages Within D Days

Ship packages **in order** using a fixed daily capacity `c`; find the minimum `c` that finishes within `D` days.

- **Search space:** `C ∈ [max(weights), sum(weights)]` — capacity can't be smaller than the heaviest single package, and shipping everything in one day is always feasible.
- `feasible(c)`: greedily load consecutive packages onto "today" until the next one would exceed `c`, then start a new day; check `days ≤ D`.

```
def ship_within_days(weights: Sequence[int], days: int) -> int:
    def feasible(capacity: int) -> bool:
        needed, current = 1, 0
        for w in weights:
            if current + w <= capacity:
                current += w
            else:
                needed += 1
                current = w
        return needed <= days

    return first_true(max(weights), sum(weights), feasible)
```

This has the exact same shape as Koko — `candidate → simulate → count → compare to a limit` — just with a different simulation.

4.3 LC 1283 — Find the Smallest Divisor Given a Threshold

Same skeleton again: minimum divisor `d` so that `sum(ceil(n/d) for n in nums) ≤ threshold`.

```
def smallest_divisor(nums: Sequence[int], threshold: int) -> int:
    def feasible(divisor: int) -> bool:
        return sum((n + divisor - 1) // divisor for n in nums) <= threshold

    return first_true(1, max(nums), feasible)
```

4.4 LC 1482 — Minimum Number of Days to Make m Bouquets

`bloomDay[i]` = day flower `i` blooms. Need `m` bouquets of `k` *adjacent* already-bloomed flowers each. Find the earliest day this is possible.

- **Search space:** `day ∈ [min(bloomDay), max(bloomDay)]`.
- **`feasible(day)`:** scan left to right counting a run of adjacent already-bloomed (`bloomDay[i] ≤ day`) flowers; every time the run hits length `k`, that's one bouquet and the run resets. Feasible once `bouquets ≥ m`.
- If `m * k > len(bloomDay)` it's impossible outright — not enough flowers even in the best case.

```
def min_days_bouquets(bloom_day: Sequence[int], m: int, k: int) -> int:
    if m * k > len(bloom_day):
        return -1

    def feasible(day: int) -> bool:
        bouquets = consecutive = 0
        for b in bloom_day:
            if b <= day:
                consecutive += 1
                if consecutive == k:
                    bouquets += 1
                    consecutive = 0
                    if bouquets >= m:
                        return True
            else:
                consecutive = 0
        return False

    return first_true(min(bloom_day), max(bloom_day), feasible)
```

4.5 LC 410 — Split Array Largest Sum

Split `nums` into **at most** `k` **contiguous** groups so the largest group sum is as small as possible.

- **Search space:** possible values of "largest allowed group sum", `x ∈ [max(nums), sum(nums)]`.

- `feasible(x)` : greedily grow the current group until the next element would push it over `x` , then start a new group; feasible when the number of groups needed is `<= k` .

```
def split_array(nums: Sequence[int], k: int) -> int:
    def feasible(limit: int) -> bool:
        groups, current = 1, 0
        for num in nums:
            if current + num <= limit:
                current += num
            else:
                groups += 1
                current = num
        return groups <= k

    return first_true(max(nums), sum(nums), feasible)
```

Dry run (`nums=[7,2,5,10,8]` , `k=2`):

left	right	mid	action
10	32	21	<code>feasible(21)=True -> right = mid</code>
10	21	15	<code>feasible(15)=False -> left = mid+1</code>
16	21	18	<code>feasible(18)=True -> right = mid</code>
16	18	17	<code>feasible(17)=False -> left = mid+1</code>

answer: minimum largest-subarray-sum 18

Notice this is *identical in structure* to the shipping problem — both ask "what's the smallest cap that keeps the number of greedily-formed groups within a budget?" Recognizing that Ship-Within-D-Days, Split-Array, and Smallest-Divisor are the same template wearing different names is exactly the transferable skill this pattern is meant to build.

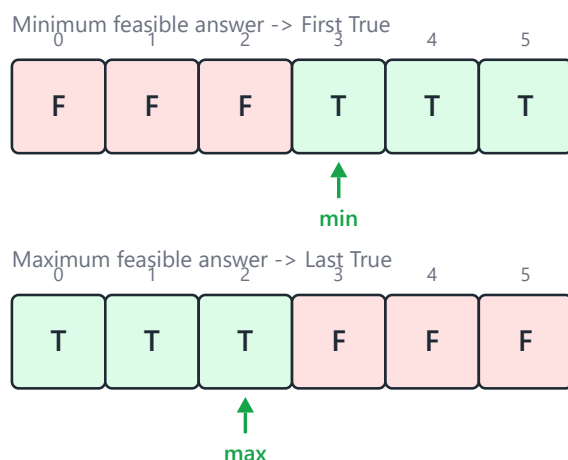
Part 5 — Maximum Feasible Answer (Last True over a value range)

Reverse the question: "*find the largest value that still satisfies a constraint.*" Same recognition test as Part 4, but feasibility now looks like `True True True False False` — we want the **last** True, so this uses the `left = mid` / upper-midpoint template from Section 2.4-2.5.

5.1 Aggressive Cows / LC 1552 — Magnetic Force Between Two Balls

Classic statement: place `cows` cows into stalls at given positions so that the **minimum distance between any two adjacent cows is as large as possible**. (LeetCode's "Magnetic Force Between Two Balls" is the identical problem with balls/baskets instead of cows/stalls.)

- **Search space:** candidate minimum distance $d \in [0, \max(\text{stalls}) - \min(\text{stalls})]$.
- **`feasible(d)`** : sort the stalls; greedily place a cow at the first stall, then repeatedly place the next cow at the earliest remaining stall that is $\geq d$ away from the last placed cow; feasible if all `cows` were placed.
- **Monotonic:** if distance `d` is achievable, so is every smaller distance — `True True True False False` — so we want the **last** True.



```
def aggressive_cows(stalls: Sequence[int], cows: int) -> int:
    """Maximum feasible minimum distance -- Last True."""
    positions = sorted(stalls)

    def feasible(distance: int) -> bool:
        count, last = 1, positions[0]
        for p in positions[1:]:
            if p - last >= distance:
                count += 1
                last = p
            if count == cows:
                return True
        return cows <= 1

    return last_true(0, positions[-1] - positions[0], feasible)
```

Dry run (stalls=[1,2,4,8,9] , cows=3):

left	right	mid	feasible(mid)	action
0	8	4	False	feasible(4)=False -> right = mid-1
0	3	2	True	feasible(2)=True -> left = mid
2	3	3	True	feasible(3)=True -> left = mid

answer: max min-distance 3

The greedy feasibility check itself is worth pausing on: *why* is placing each cow at the *earliest* possible valid stall optimal for testing feasibility? Because delaying a placement can only ever make the *next* placement harder (less room to its right), never easier — so the greedy choice never costs you a placement you could have made. This "does greedy achieve the count" style feasibility check recurs constantly in maximum feasible answer problems.

5.2 LC 1300 — Sum of Mutated Array Closest to Target

Replace every element larger than some cap `x` with `x`; find the `x` that makes the resulting sum as close as possible to `target` (ties broken toward the smaller `x`).

- **Search space:** `x ∈ [0, max(arr)]`.

- **Observation:** `summed(x) = sum(min(v, x) for v in arr)` is monotonically non-decreasing in `x`. So `summed(x) >= target` is a monotonic predicate — First True gives the smallest `x` whose sum reaches or passes `target`. The actual answer is then whichever of that boundary and its immediate predecessor lands closer to `target`.

```
def find_best_value(arr: Sequence[int], target: int) -> int:
    """Binary search the answer value in [0, max(arr)]; take whichever of the
    two boundary candidates gives a sum closer to target (ties -> smaller)."""
    def summed(cap: int) -> int:
        return sum(min(x, cap) for x in arr)

    lo, hi = 0, max(arr)
    ans = first_true(lo, hi, lambda cap: summed(cap) >= target)
    if ans == lo:
        return ans
    lower_sum = summed(ans - 1)
    upper_sum = summed(ans)
    if target - lower_sum <= upper_sum - target:
        return ans - 1
    return ans
```

This is a useful variant to recognize: the *feasibility predicate itself* is First True, but the **problem's real answer** needs one extra comparison around that boundary because "closest to target" isn't quite the same question as "first value that reaches target."

Part 6 — Square Roots: Integer and Floating-Point

6.1 LC 69 — Integer Square Root (Last True)

`floor(sqrt(x))` for `x = 27` is `5`, because `52 = 25 ≤ 27` but `62 = 36 > 27`. There is no array — the search space is the possible integer answers `[0, x]`, and the predicate `mid*mid ≤ x` is monotonic: `True True True True True False False`. We want the **last** True.

```
def integer_sqrt(x: int) -> int:
    if x < 2:
        return x
    return last_true(0, x, lambda mid: mid * mid <= x)
```

Dry run (`x = 27`):

left	right	mid	mid ²	action
0	27	14	196	14 ² =196 ≤ 27? False -> right = mid-1
0	13	7	49	7 ² =49 ≤ 27? False -> right = mid-1
0	6	3	9	3 ² =9 ≤ 27? True -> left = mid
3	6	5	25	5 ² =25 ≤ 27? True -> left = mid
5	6	6	36	6 ² =36 ≤ 27? False -> right = mid-1

answer: `floor(sqrt(27)) = 5`

The generalizing insight: **binary search doesn't need a sorted array at all** — it only needs a monotonic function. `f(m) = m2` is monotonically increasing for `m ≥ 0`, and that alone is enough structure to binary search over it. This same idea — binary search on a monotonic *mathematical* function rather than a data structure — is what makes every "binary search on the answer" problem in Parts 3–5 work.

6.2 Floating-point binary search

Now suppose the answer isn't an integer — $\text{sqrt}(2) \approx 1.41421356\dots$. The mental model is identical (search space $\rightarrow$ midpoint $\rightarrow$ evaluate $\rightarrow$ eliminate half), but the stopping condition changes: `left` and `right` will *approach* each other but may never become exactly equal as floats. Instead, stop once the interval is smaller than the required precision.

Searching for `sqrt(2)`: the interval shrinks instead of hitting `left == right`

<code>[1.0, 2.0]</code>	$\rightarrow \text{mid}=1.5, 1.5^2=2.25 > 2$, go left
<code>[1.25, 1.5]</code>	$\rightarrow \text{mid}=1.375$ too small, go right
<code>[1.375, 1.5]</code>	$\rightarrow$ keeps halving...
<code>[1.4140, 1.4143]</code>	$\rightarrow$ stop when <code>right-left < precision</code>

```
def sqrt_precision(x: float, precision: float = 1e-7) -> float:
    """Floating-point variant: shrink the interval instead of using left==right."""
    if x < 1:
        left, right = 0.0, 1.0
    else:
        left, right = 0.0, float(x)
    while right - left > precision:
        mid = (left + right) / 2
        if mid * mid < x:
            left = mid
        else:
            right = mid
    return (left + right) / 2
```

Two practical notes:

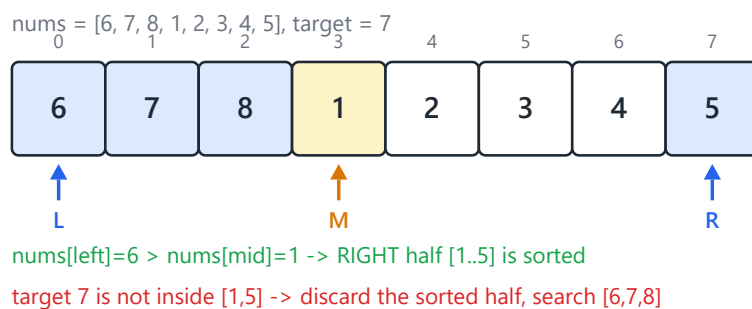
- Some implementations run a **fixed number of iterations** (e.g. `for _ in range(100)`) instead of checking `right - left`. Both work — each iteration still halves the interval, so a fixed iteration count that's comfortably larger than `log2(range / precision)` reaches the same precision.
- Never compare floats with `==`. The stopping test is always "is the interval small enough," never "did I land exactly on the answer."

Part 7 — Rotated Sorted Arrays

7.1 LC 33 — Search in Rotated Sorted Array

[6, 7, 8, 1, 2, 3, 4, 5] was originally sorted, then rotated around some pivot. The array as a whole is *not* sorted, so the Part 1 invariant ("everything left of mid is smaller") no longer holds globally. But a weaker, still-useful property survives:

At every midpoint, at least one of the two halves is fully sorted.



The algorithm: find *which* half is sorted (`nums[left] <= nums[mid]` means the left half is), then ask whether `target` could be inside that sorted half's value range. If yes, recurse there; if no, the target — if it exists at all — must be in the other half, so discard the sorted one.

```
def search_rotated(nums: Sequence[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]: # left half sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else: # right half sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

Dry run (nums=[6,7,8,1,2,3,4,5] , target=7):

left	right	mid	nums[mid]	action
0	7	3	1	right half sorted (nums[mid]=1<=nums[right]=5); target in (1,5]? False -> right=mid-1
0	2	1	7	nums[mid]=7 == target -> return 1

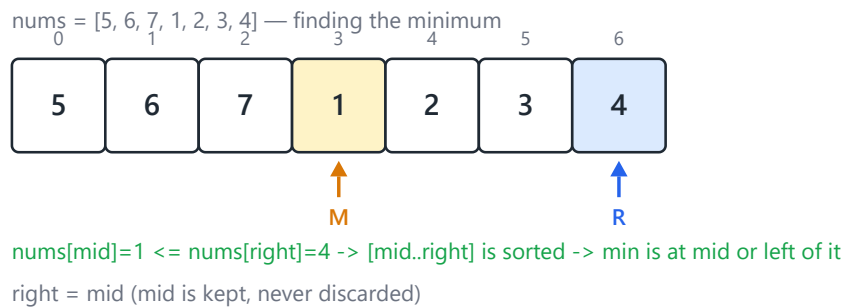
answer: index 1

Recognition signal: "sorted array, rotated/pivoted, find a target." Decision rule at every step:

1. Compute mid.
2. nums[mid] == target? -> found
3. Is nums[left] <= nums[mid]? -> left half is sorted
4. target in [nums[left], nums[mid]]? -> search left half
5. otherwise -> search right half
6. Otherwise right half is sorted; symmetric check.

7.2 LC 153 — Find Minimum in Rotated Sorted Array

No target this time — just find the minimum. Compare `nums[mid]` against `nums[right]` (not `nums[left]` — the right endpoint is what tells us whether we've crossed the rotation point):



- `nums[mid] > nums[right]` → the rotation point (and therefore the minimum) is strictly to the right of `mid` → `left = mid + 1`.
- `nums[mid] <= nums[right]` → `[mid, right]` is already sorted, so its minimum is `nums[mid]` itself → `mid` **cannot be discarded** → `right = mid`.

Because one branch keeps `mid`, this is boundary-search style: `left < right` with the lower midpoint (the branch that keeps `mid` moves `right` down).

```
def find_min_rotated(nums: Sequence[int]) -> int:
    left, right = 0, len(nums) - 1
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] > nums[right]:
            left = mid + 1
        else:
            right = mid
    return nums[left]
```

Dry run (`nums=[5,6,7,1,2,3,4]`):

left	right	mid	nums[mid]	action
0	6	3	1	nums[mid]=1 > nums[right]=4? False -> right=mid
0	3	1	6	nums[mid]=6 > nums[right]=1? True -> left=mid+1

left	right	mid	nums[mid]	action
2	3	2	7	nums[mid]=7 > nums[right]=1? True -> left=mid+1

answer: minimum 1

7.3 LC 81 — Search in Rotated Sorted Array II (with duplicates)

Duplicates break the "which half is sorted" test: if `nums[left] == nums[mid] == nums[right]`, both sides *look* identical and we genuinely cannot tell which is sorted. The fix is to shrink defensively by one from each end and try again — this can degrade the worst case to $O(n)$ (e.g. `[2,2,2,2,2,2,1,2,2]`), but the common case stays logarithmic.

```
def search_rotated_duplicates(nums: Sequence[int], target: int) -> bool:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target:
            return True
        if nums[left] == nums[mid] == nums[right]:
            left += 1
            right -= 1
        elif nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return False
```

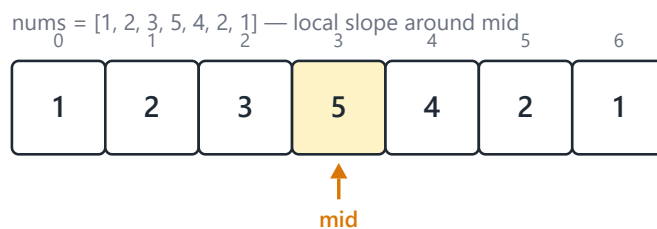
Common mistake

Applying the LC 33 rotated-search code unmodified to an array with duplicates. It will occasionally return the wrong "sorted half" and either miss the target or loop forever on adversarial inputs — always add the three-way-equal check first when duplicates are possible.

Part 8 — Peak / Local Structure

8.1 LC 162 — Find Peak Element

`nums = [1, 2, 3, 5, 4, 2, 1]`. A peak is any element strictly greater than both neighbors (edges are compared against `-infinity`). The array **is not sorted**, and yet binary search still applies, because we don't need global order — only enough local structure to prove one side must contain a valid answer.



`nums[mid]=5 > nums[mid+1]=4 -> downhill -> a peak is at mid or to its left`

`right = mid` (uphill instead would give `left = mid+1`)

Look at the slope between `mid` and `mid + 1`:

- `nums[mid] < nums[mid+1]` (**uphill**) → walking further right, the sequence either keeps climbing until the array ends (the last element is then a peak, since nothing follows it) or eventually turns downward (the turning point is a peak). Either way, **a peak is guaranteed to exist to the right** → `left = mid + 1` (mid is provably not a peak, safe to drop).
- `nums[mid] > nums[mid+1]` (**downhill**) → a peak exists at `mid` or somewhere to its left → `mid` **cannot be discarded** → `right = mid`.

```
def find_peak_element(nums: Sequence[int]) -> int:
    left, right = 0, len(nums) - 1
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left
```

Dry run (nums=[1,2,3,5,4,2,1]):

left	right	mid	nums[mid]	action
0	6	3	5	nums[mid]=5 < nums[mid+1]=4? False -> right=mid (downhill)
0	3	1	2	nums[mid]=2 < nums[mid+1]=3? True -> left=mid+1 (uphill)
2	3	2	3	nums[mid]=3 < nums[mid+1]=5? True -> left=mid+1 (uphill)

answer: index 3 (value 5)

Proving termination: the invariant is *"there is always at least one peak inside [left, right]"* — true initially for the whole array, and every update preserves it (uphill: right side still guaranteed to hold one; downhill: left side, including mid, still guaranteed to hold one). When left == right, the guaranteed region has shrunk to a single element, so that element must be the peak. This is the same proof technique as the [left, right] invariant from Part 1 — just applied to "contains a peak" instead of "contains the target."

8.2 LC 852 — Peak Index in a Mountain Array

A "mountain array" strictly increases then strictly decreases, with exactly one peak — a special case of Section 8.1 where the peak is guaranteed unique. The exact same recurrence applies:

```
def peak_index_mountain(arr: Sequence[int]) -> int:
    # Same recurrence as find_peak_element; mountain guarantees uniqueness.
    return find_peak_element(arr)
```

8.3 LC 1095 — Find in Mountain Array (bonus)

A harder variant behind an *opaque* `MountainArray` interface — each `.get(index)` call is metered, so a correct solution must be careful not to re-probe indices. The approach composes three binary searches:

1. **Find the peak** using the same uphill/downhill recurrence as 8.1/8.2, but reading through `.get()` instead of a real array.
2. **Binary search the ascending half** `[0, peak]` for the target (ordinary exact search).
3. If not found, **binary search the descending half** `[peak, n-1]` (comparisons flip direction since this half is decreasing).

```
def find_in_mountain_array(target: int, mountain_arr: MountainArray) -> int:
    n = mountain_arr.length()
    peak = last_true(0, n - 1, lambda i: i == 0 or mountain_arr.get(i) > mountain_arr.get(i + 1))

    # search ascending half [0, peak] for target
    left, right = 0, peak
    while left <= right:
        mid = left + (right - left) // 2
        v = mountain_arr.get(mid)
        if v == target:
            return mid
        elif v < target:
            left = mid + 1
        else:
            right = mid - 1

    # search descending half [peak, n-1] for target
    left, right = peak, n - 1
    while left <= right:
        mid = left + (right - left) // 2
        v = mountain_arr.get(mid)
        if v == target:
            return mid
        elif v > target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Part 9 — Binary Search Over 2D Search Spaces

Matrix problems split into two genuinely different families that are easy to conflate. **Do not assume the same strategy works for both.**

9.1 LC 74 — Search a 2D Matrix (globally sorted)

```
1   3   5   7
10  11  16  20
23  30  34  60
```

Here every row is sorted, **and** each row's first element is greater than the previous row's last element — so the whole matrix, if flattened row-major, is one sorted array. We don't need to actually build that array: a logical index `mid` in `[0, rows*cols - 1]` maps back with `row = mid // cols`, `col = mid % cols`, and ordinary Pattern 1 search runs on top of that mapping.

Globally sorted matrix — treat it as one flattened sorted array

1	3	5	7
10	11	16	20
23	30	34	60

`row = mid // cols`, `col = mid % cols` -> ordinary binary search over `[0, rows*cols-1]`

```
def search_matrix(matrix: Sequence[Sequence[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
        mid = left + (right - left) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        elif value < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

Complexity: $O(\log(\text{rows} \times \text{cols}))$.

9.2 LC 240 — Search a 2D Matrix II (row and column sorted, not globally)

```
1  4  7  11
2  5  8  12
3  6  9  16
10 13 14 20
```

Rows are sorted left→right *and* columns are sorted top→bottom, but the matrix is **not** globally sorted (e.g. `10 < 11` yet `10` sits below `11`). The flattening trick from 9.1 no longer applies. Instead, start at the **top-right corner** and walk:

- `current > target` → everything **below** the current cell in this column is even larger (column is sorted downward) → move **left**.
- `current < target` → everything to the **left** of the current cell in this row is even smaller (row is sorted left-to-right) → move **down**.

start

1	4	7	11
2	5	8	12
3	6	9	16
10	13	14	20

target=9: 11>9 move left | 7<9 move down | 8<9 move down |
each step eliminates one full row OR column -> O(rows+cols)

```
def search_matrix_ii(matrix: Sequence[Sequence[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False
    rows, cols = len(matrix), len(matrix[0])
    row, col = 0, cols - 1
    while row < rows and col >= 0:
        value = matrix[row][col]
        if value == target:
            return True
        elif value > target:
            col -= 1
        else:
            row += 1
    return False
```

This is called a **staircase search**. It is *not* literally binary search — at each step it eliminates one entire row **or** one entire column, not half the remaining space — giving $O(\text{rows} + \text{cols})$ rather than $O(\log(\text{rows} \cdot \text{cols}))$. It's included here because it's frequently confused with binary search on a matrix; recognizing that it's a different technique (same underlying idea — use an ordering to eliminate a region — different mechanics) is itself the useful skill.

A correction worked through live in the source lesson

Correction worked through live in the source chat

Row+col sorted matrix, at cell 11, target 9, column above is [9]

1	4	7	9
2	5	8	11
3	6	12	13

Wrong: "move left eliminates the whole column" -- it does not: 9 sits ABOVE 11 in that column.

While deriving the staircase search, the first explanation offered was "moving left eliminates the entire current column" — which is **wrong**. Standing at value 11 with target 9, if 9 happens to sit *above* 11 in that same column, moving left must not throw it away. The correct statement, after the learner pushed back on exactly this point, is narrower: moving left only eliminates the cells **below** the current cell in that column (they're proven larger because the column is sorted downward and $\text{current} > \text{target}$) — cells above the current position are simply untouched, not eliminated. Starting at the top-right corner sidesteps the issue on the very first move (nothing is above the start), and the same logic holds at every subsequent step. This is a good example of why the "what can I *prove* is impossible?" framing (Part 12) is safer than a vaguer "eliminate that side" intuition.

9.3 LC 378 — Kth Smallest Element in a Sorted Matrix

Binary search the **value range**, not indices or a row/column position — same family as Part 3's answer-space search, applied to a matrix. Search space: $x \in [\text{matrix}[0][0], \text{matrix}[-1][-1]]$. For a candidate x , count how many matrix entries are $\leq x$ using the

same staircase walk as 9.2 (start bottom-left; move right while advancing the count when a value qualifies, otherwise move up) — that count is $O(n)$ per probe. Binary search for the first x where $\text{count}(x) \geq k$.

```
def kth_smallest_matrix(matrix: Sequence[Sequence[int]], k: int) -> int:
    n = len(matrix)

    def count_less_equal(x: int) -> int:
        # staircase count from bottom-left
        count, row, col = 0, n - 1, 0
        while row >= 0 and col < n:
            if matrix[row][col] <= x:
                count += row + 1
                col += 1
            else:
                row -= 1
        return count

    return first_true(matrix[0][0], matrix[-1][-1], lambda mid: count_less_equal(mid) >= k)
```

Complexity: $O(n \log(\max - \min))$, where n is the matrix side length.

Part 10 — More Boundary & Value-Range Problems

The source lesson closed by listing several "easy → medium" problems as practice extensions without deriving them in full. Each one is solved and tested here, classified by which template it actually is.

10.1 LC 35 — Search Insert Position

"Where would `target` go to keep the array sorted?" This is nothing but `lower_bound` under a different name.

```
def search_insert(nums: Sequence[int], target: int) -> int:
    return lower_bound(nums, target)
```

10.2 LC 34 — Find First and Last Position of Element in Sorted Array

Direct application of the lower-bound/upper-bound relationship from Section 2.3: first position is `lower_bound(target)`, last is `upper_bound(target) - 1`, and if the target isn't present the lower bound won't actually equal it.

```
def search_range(nums: Sequence[int], target: int) -> Tuple[int, int]:
    lb = lower_bound(nums, target)
    if lb == len(nums) or nums[lb] != target:
        return (-1, -1)
    ub = upper_bound(nums, target)
    return (lb, ub - 1)
```

10.3 LC 278 — First Bad Version

`isBadVersion(v)` is a monotonic predicate (`False...False True...True`) — textbook First True over `[1, n]` .

```
def first_bad_version(n: int, is_bad_version: Callable[[int], bool]) -> int:
    return first_true(1, n, is_bad_version)
```

10.4 LC 744 — Find Smallest Letter Greater Than Target

Upper bound over character codes, with one twist: if `target` is `>=` every letter, the answer **wraps around** to `letters[0]` .

```
def next_greatest_letter(letters: Sequence[str], target: str) -> str:
    idx = upper_bound([ord(c) for c in letters], ord(target))
    return letters[idx % len(letters)]
```

10.5 LC 540 — Single Element in a Sorted Array

Every value appears exactly twice except one, and the array is sorted, so before the single element every pair sits at `(even, odd)` index positions; after it, the parity flips. Restrict `mid` to even indices and binary search for *where the parity breaks* — another First True in disguise, this time on the predicate "has the pairing already been disrupted by index `i` ?"

```
def single_non_duplicate(nums: Sequence[int]) -> int:
    """Every element appears twice except one; pairs sit at (even, odd) index
    positions until the single element shifts the parity. First True on:
    'has the parity broken by (or before) index i?' using only even indices."""
    left, right = 0, len(nums) - 1
    while left < right:
        mid = left + (right - left) // 2
        if mid % 2 == 1:
            mid -= 1 # force mid to an even index
        if nums[mid] == nums[mid + 1]:
            # pair intact up to mid+1 -> single element is further right
            left = mid + 2
        else:
            right = mid
    return nums[left]
```

10.6 LC 287 — Find the Duplicate Number

`nums` holds `n + 1` integers in `[1, n]`, exactly one of which repeats. (Floyd's cycle detection is the classic $O(1)$ -space answer; the binary search below is included because it's a clean example of **binary searching the answer's value range on an unsorted array** — the array's order is irrelevant, only counts matter.)

- **Search space:** candidate values `[1, n]`.
- **`feasible(mid)`:** `count(x <= mid for x in nums) > mid`. By the pigeonhole principle, if this count ever exceeds `mid`, the duplicate must be `<= mid` (there are more values crammed into `[1, mid]` than there are slots). Monotonic once True.

```
def find_duplicate(nums: Sequence[int]) -> int:
    """nums has n+1 ints in [1, n]. Search space = candidate values [1, n],
    not indices. feasible(mid) = 'count(<=mid) > mid' is monotonic once True."""
    n = len(nums) - 1

    def too_many_below_or_at(mid: int) -> bool:
        return sum(1 for x in nums if x <= mid) > mid

    return first_true(1, n, too_many_below_or_at)
```

10.7 LC 1539 — Kth Missing Positive Number

`missing(i) = arr[i] - (i + 1)` counts how many positive integers have been skipped by the time we reach index `i` (if none were skipped, `arr[i]` would equal `i + 1`). This count is monotonic non-decreasing, so First True on `missing(i) >= k` finds the transition point directly.

```
def kth_missing_positive(arr: Sequence[int], k: int) -> int:
    """missing(i) = arr[i] - (i + 1) counts positives skipped by index i.
    missing(...) is monotonic non-decreasing -> First True on missing(i) >= k."""
    n = len(arr)
    if arr[n - 1] - n < k:
        return n + k
    idx = first_true(0, n - 1, lambda i: arr[i] - (i + 1) >= k)
    return idx + k
```

10.8 LC 1385 — Find the Distance Value Between Two Arrays

For each `a` in `arr1`, count it only if **no** element of `arr2` is within `d`. Sort `arr2` once, then for each `a` use `lower_bound(a - d)` and check whether that position's value already exceeds `a + d` — an $O(n \log m)$ alternative to the $O(n \cdot m)$ brute force.

```
def distance_value(arr1: Sequence[int], arr2: Sequence[int], d: int) -> int:
    sorted2 = sorted(arr2)
    count = 0
    for a in arr1:
        lo = lower_bound(sorted2, a - d)
        if lo == len(sorted2) or sorted2[lo] > a + d:
            count += 1
    return count
```

10.9 LC 1150 — Check If a Number Is Majority Element in a Sorted Array (bonus)

Another direct application of Section 2.3's count formula: a value is a majority element iff

```
upper_bound(v) - lower_bound(v) > n // 2.
```

```
def is_majority_element(nums: Sequence[int], target: int) -> bool:
    lb = lower_bound(nums, target)
    ub = upper_bound(nums, target)
    return (ub - lb) > len(nums) // 2
```

10.10 LC 981 — Time Based Key-Value Store

Not a single binary search but a small design problem built entirely out of one: each key stores an append-only, timestamp-sorted list of `(timestamp, value)` pairs (LeetCode guarantees `set` calls arrive in non-decreasing timestamp order per key), and `get(key, timestamp)` needs the value from the **latest set at or before** `timestamp` — exactly `upper_bound(timestamp) - 1` over that key's timestamp list.

```
class TimeMap:
    def __init__(self):
        self._store: dict[str, List[Tuple[int, str]]] = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        self._store.setdefault(key, []).append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        entries = self._store.get(key)
        if not entries:
            return ""
        stamps = [t for t, _ in entries]
        # last index with stamps[i] <= timestamp == upper_bound(timestamp) - 1
        idx = upper_bound(stamps, timestamp) - 1
        return entries[idx][1] if idx >= 0 else ""
```

10.11 LC 528 — Random Pick with Weight

Build a prefix-sum array of the weights; a uniformly random integer `x` in `[1, total]` lands in exactly one weighted "bucket," found with `lower_bound(prefix, x)`. The randomness is a thin wrapper around a deterministic binary search, which is what actually gets tested.

```
class RandomPickWeight:
    def __init__(self, w: Sequence[int]):
        self.prefix: List[int] = []
        total = 0
        for x in w:
            total += x
            self.prefix.append(total)
        self.total = total

    def index_for(self, x: int) -> int:
        """Deterministic helper (used by tests): the index whose weighted
        bucket contains value x in [1, total] -- first prefix[i] >= x."""
        return lower_bound(self.prefix, x)

    def pickIndex(self) -> int: # noqa: N802 (LeetCode's required name)
        x = random.randint(1, self.total)
        return self.index_for(x)
```

Part 11 — LC 4: Median of Two Sorted Arrays

The hardest problem the source lesson names, and worth isolating because the search space is neither "array indices" nor "answer values" — it's a **partition point**.

11.1 Reframing the problem

Merging two sorted arrays and reading off the middle is $O(m+n)$; the required complexity is $O(\log(\min(m, n)))$. To get there, stop thinking "find the median value" and think "**find a way to cut both arrays so that everything on the left of both cuts is $\leq$ everything on the right of both cuts, with the correct total count on the left.**"

Call the smaller array `nums1` (length `m`) and the larger `nums2` (length `n`). If we place the cut in `nums1` after `i` elements, the cut in `nums2` is *forced*: it must be after `j = half - i` elements, where `half = (m + n + 1) // 2` is exactly how many elements belong on the left side in total (this one line is why we always search the **smaller** array — `i` has to range over `[0, m]`, and pairing it with a forced `j` keeps `j` automatically in-bounds for `nums2`).

11.2 The search space is `i`, the partition count

- **Search space:** `i ∈ [0, m]` — how many elements of `nums1` sit on the left side. Binary search over indices again, but the meaning of the index has changed: it's a *cut position*, not an element to compare against a target.
- `mid` = a **candidate cut position** `i`.
- **Condition to check at** `mid`: define the four boundary values around both cuts (using `-infinity` / `+infinity` when a cut lands at an array edge):

```
left1  = nums1[i-1]  (or -inf if i == 0)
right1 = nums1[i]    (or +inf if i == m)
left2  = nums2[j-1]  (or -inf if j == 0)
right2 = nums2[j]     (or +inf if j == n)
```

The partition is valid exactly when `left1 <= right2` **and** `left2 <= right1`. If `left1 > right2`, `i` is too large (shift the cut left, `right = i - 1`); if `left2 > right1`, `i` is too small (`left = i + 1`). This is a monotonic feasibility test, so it's still ordinary binary search — the "found" condition just replaces `==`.

Once a valid partition is found: odd total length → the median is `max(left1, left2)`; even → the average of `max(left1, left2)` and `min(right1, right2)`.

```
def find_median_sorted_arrays(nums1: Sequence[int], nums2: Sequence[int]) -> float:
    """Binary search the PARTITION point in the smaller array. Search space is
    'how many elements of nums1 go on the left side', not array values."""
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    left, right = 0, m
    half = (m + n + 1) // 2
    NEG, POS = float("-inf"), float("inf")

    while left <= right:
        i = left + (right - left) // 2
        j = half - i

        left1 = nums1[i - 1] if i > 0 else NEG
        right1 = nums1[i] if i < m else POS
        left2 = nums2[j - 1] if j > 0 else NEG
        right2 = nums2[j] if j < n else POS

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return float(max(left1, left2))
            return (max(left1, left2) + min(right1, right2)) / 2.0
        elif left1 > right2:
            right = i - 1
        else:
            left = i + 1
    raise ValueError("input arrays are not sorted")
```

Why this belongs in a binary search guide and not a "clever trick" collection: the recognition questions from Part 1 still apply verbatim — *what is my search space* (a partition count), *is it ordered* (yes — moving the cut right only shifts elements left→right across it), *can I eliminate half after inspecting one candidate* (yes — an invalid partition tells you unambiguously which direction to move). The problem looks unrelated to `[1,3,5,7,9]` -style search only until you ask those same four questions.

Part 12 — Recognition, and When Binary Search Does *Not* Apply

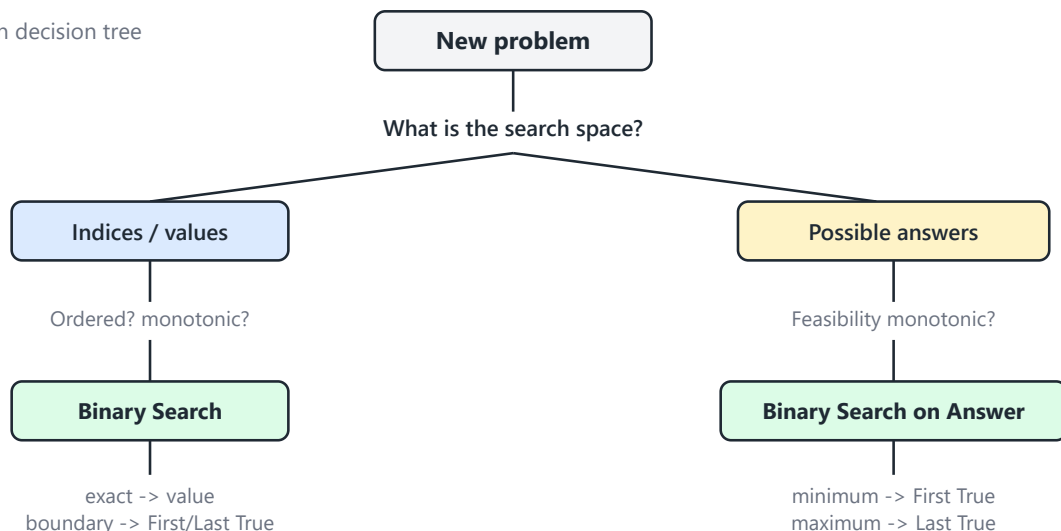
12.1 The five-question checklist

Before reaching for binary search on an unfamiliar problem, ask, in order:

1. **What is my search space?** Indices, values, or possible answers?
2. **Is there an ordering or monotonic property over that space?**
3. **Can I evaluate one candidate efficiently** (ideally without touching every other candidate)?
4. **If I learn something about one candidate, can I eliminate an entire half of the remaining space?**
5. **What exactly am I looking for** — an exact value, a first/last True boundary, a minimum feasible answer, or a maximum feasible answer?

If the answer to (2) and (4) is yes, binary search is a strong candidate, and (5) tells you which of the two templates from Section 2.4 to use.

Recognition decision tree



12.2 Binary search vs. other techniques

Don't reach for binary search just because a problem involves numbers or the word "sorted." Compare against the alternatives explicitly:

Problem	Right tool	Why not binary search
Do two numbers in an unsorted array sum to <code>K</code> ?	Hash map, <code>O(n)</code>	No ordering to exploit; nothing to eliminate by inspecting one element
Longest subarray with sum <code><= K</code>	Sliding window	Maintaining a contiguous, expanding/shrinking window isn't "eliminate half a search space"
Two sorted-but-separate pointers converging	Two pointers	Two <i>moving</i> positions, not one shrinking <code>[left, right]</code> range
One-off search over unsorted data	Just scan, <code>O(n)</code>	Sorting first costs <code>O(n log n)</code> — more expensive than the one linear scan it was meant to avoid
Repeated queries over the same data	Sort once, <code>O(n log n)</code> , then binary search each query in <code>O(log n)</code>	Now the sort cost is amortized across many searches, which <i>does</i> pay off

```
def two_sum_unsorted(nums: Sequence[int], target: int) -> Optional[Tuple[int, int]]:
    """Not binary search: nums is unsorted, so there is no ordering to halve.
    A hash map is the right tool -- O(n) time. Included so the guide can show
    a case where reaching for binary search would be a mistake."""
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i
    return None
```

The habit worth building: **ask "what does binary search actually buy me here?"** If there's no way to prove that inspecting one middle element eliminates a provable half of what remains, binary search doesn't apply — reach for the technique whose elimination argument *does* hold.

12.3 Common mistakes checklist

- Using binary search on data with **no monotonic/ordered property** at all.
- Wrong search space — e.g. treating a rotated array like a fully sorted one.
- Wrong boundary — using lower bound when the problem actually needs upper bound (or vice versa); confusing "first True" with "last True."
- Off-by-one boundary updates: forgetting that `nums[mid] < target` also eliminates `mid`, not just everything before it.
- Mixing `left <= right` (exact search) with boundary-search branches that *keep* `mid` — this produces either an infinite loop or a wrong answer.
- Using the ordinary (lower) midpoint on a Last True search — see Section 2.5.
- Returning `mid` when the real answer is a *boundary* (`left`), not whichever index the loop happened to stop on.
- Forgetting duplicates can break the "which half is sorted" test on rotated arrays (Part 7.3).
- Using floating-point equality (`==`) instead of an interval/precision check.
- Assuming a rotated array behaves like a normally sorted one (Part 7).
- Assuming a row+column sorted matrix behaves like a globally sorted one (Part 9) — and the column-elimination correction in Section 9.2 shows how easy that particular mistake is to make even when explaining it carefully.
- Not stating, out loud, what `mid` represents before writing any code. Every derivation in this guide starts with search space → `mid` 's meaning → predicate, in that order, for exactly this reason.

Part 13 — Templates & Final Decision Guide

13.1 The four skeletons worth memorizing

Everything in this guide compiles down to these four. Memorize the *shapes*, not fifteen separate problem solutions.

1. Exact search

```
while left <= right:
    mid = left + (right - left) // 2
    if found: return mid
    elif too_small: left = mid + 1
    else: right = mid - 1
```

2. First True / Minimum feasible

```
while left < right:
    mid = left + (right - left) // 2
    if feasible(mid): right = mid
    else: left = mid + 1
return left
```

3. Last True / Maximum feasible

```
while left < right:
    mid = left + (right - left + 1) // 2 # upper mid
    if feasible(mid): left = mid
    else: right = mid - 1
return left
```

4. Rotated / structural binary search — there is no single skeleton. Instead: inspect `mid` → identify which structure is provably intact (a sorted half, a guaranteed-uphill region, a guaranteed-downhill region) → determine whether the target/answer could exist there → discard the other region.

13.2 Final decision table

Problem signal	Think
Sorted array + exact target	Standard binary search (template 1)
First position satisfying a condition	First True / lower bound (template 2)
Last position satisfying a condition	Last True / upper bound (template 3)
Duplicates + a boundary to find	Lower/upper bound, <code>count = UB - LB</code>
"Minimum X such that..."	Binary search on answer, First True
"Maximum X such that..."	Binary search on answer, Last True
Feasibility changes monotonically as a candidate grows	Answer-space binary search
Rotated sorted array	Identify which half is sorted (template 4)
Local increasing/decreasing structure, no global order	Peak binary search (template 4)
Matrix, globally sorted if flattened	Flatten logically, template 1
Matrix, row-sorted and column-sorted only	Staircase search — not literally binary search
Floating-point answer	Precision binary search (shrink the interval)
Two arrays, need a combined partition	Binary search the cut point (Part 11)
No ordering / no monotonicity anywhere	Binary search probably doesn't apply

13.3 Reference card — every pattern in one place

Pattern	Search space	mid means	Boundary found	Loop / midpoint
Exact search	indices	candidate index	exact match	<code>left <= right</code> , lower mid
Lower bound	indices	candidate boundary	first <code>>= target</code>	<code>left < right</code> , lower mid

Pattern	Search space	mid means	Boundary found	Loop / midpoint
Upper bound	indices	candidate boundary	first $>$ target	$left < right$, lower mid
Min feasible (Koko, Ship, Bouquets, Split, Divisor)	answer values	candidate answer	First True	$left < right$, lower mid
Max feasible (Cows/Magnetic Force, Closest Sum)	answer values	candidate answer	Last True	$left < right$, upper mid
Integer sqrt	answer values	candidate root	Last True	$left < right$, upper mid
Float sqrt	real interval	candidate root	precision-bounded	shrink interval
Rotated search	indices	candidate index	exact match via sorted half	$left \leq right$, lower mid
Find min rotated	indices	candidate index	min via sorted-half check	$left < right$, lower mid
Peak element	indices	candidate index	local max via slope	$left < right$, lower mid
2D global matrix	flattened indices	candidate index	exact match	$left \leq right$, lower mid
2D staircase	row & col position	current cell	exact match	not binary search ($O(m+n)$)
Kth smallest matrix	answer values	candidate value	First True on count	$left < right$, lower mid
Median of two arrays	partition count	candidate cut i	valid partition	$left \leq right$, lower mid

Part 14 — Recognition Drills

For each prompt below, answer the five questions from Section 12.1 **before** reading the answer key: search space, ordering/monotonicity, evaluation cost, what gets eliminated, and what exactly you're finding. Some are deliberately *not* binary search.

1. Given a sorted array of distinct integers, find the smallest index `i` such that `nums[i] >= 10`.
2. You have `n` jobs; a worker completes `k` units/day. Find the minimum `k` to finish all jobs within `d` days.
3. Given an unsorted array, determine whether any two elements sum to `k`.
4. Find the longest subarray whose sum is `<= k`, in an array of positive integers.
5. A function `f(x)` is known to be monotonically increasing (but you can't see its formula, only call it). Find the smallest `x` with `f(x) >= 100`.
6. Given a sorted array that was rotated an unknown number of times, and which **may contain duplicates**, determine whether `target` exists.
7. Given `n` ropes of different lengths, find the minimum cost to connect them all into one rope (cost of connecting two ropes = sum of their lengths).
8. Given a mountain-shaped array (strictly increasing then strictly decreasing), find the index of the maximum.
9. You must cut a log into `k` equal pieces of maximum possible integer length, given `n` logs of varying lengths. Find that maximum length.
10. Given two sorted linked lists (not arrays — no random access), find their median in $O(\log(m+n))$.

Answer key

1. **Binary search — lower bound.** Search space: indices. Predicate `nums[i] >= 10` is monotonic. First True.
2. **Binary search on the answer — minimum feasible.** Search space: possible `k` values. `feasible(k)` = can all jobs finish in `d` days at rate `k`; monotonic because larger `k` can only help. First True.
3. **Not binary search — hash map, $O(n)$.** The array is unsorted; there's no ordering to eliminate half of by inspecting one element.

4. **Not binary search — sliding window.** You're maintaining one contiguous, expanding/shrinking window, not repeatedly halving a search space. (If the array could contain negatives, even sliding window wouldn't directly apply, since window sums would stop being monotonic.)
5. **Binary search — First True**, exactly like Part 3, except the monotonic function is opaque rather than array-backed. Search space: possible `x` values; `mid` = candidate `x`; condition = `f(mid) >= 100`.
6. **Binary search — rotated search with the duplicate-defense check** (Part 7.3): when `nums[left] == nums[mid] == nums[right]`, shrink both ends by one and continue.
7. **Not this guide's pattern — greedy + min-heap.** There's no monotonic predicate to binary search over; connecting the two smallest ropes first is a greedy exchange argument, not a search-space elimination argument. (Included as a reminder that "involves picking a minimum" doesn't automatically mean binary search.)
8. **Binary search — peak / local structure** (Part 8), using the uphill/downhill slope test; a mountain array guarantees a unique peak.
9. **Binary search on the answer — maximum feasible** (Part 5's family): search space is possible piece lengths; `feasible(length)` = total pieces obtainable at that length `>= k`; feasibility is `True...True False...False` as length grows, so Last True.
10. **Binary search — but the *search space* has to change.** Linked lists have no random access, so index-based `mid` doesn't work directly; the practical approach converts to the same partition-search idea as Part 11 (Median of Two Sorted Arrays) by binary searching a partition *count* and walking pointers `count` steps rather than jumping to an index — the reasoning about *what* to search transfers even when *how* you access the middle has to change.

Part 15 — Interview Q&A

Short, direct answers you should be able to give without hesitation.

Q: Why is binary search $O(\log n)$? Each iteration keeps roughly half of the remaining search space, so after k iterations $n/2^k$ candidates remain; setting that to 1 gives $k \approx \log_2 n$.

Q: When do you use $left \leq right$ vs. $left < right$? $left \leq right$ when the last remaining candidate still needs to be explicitly checked (exact-value search — mid gets eliminated every branch). $left < right$ when you're narrowing toward one candidate and an invariant already guarantees that candidate is the answer (boundary/answer search — one branch may keep mid).

Q: Why does Last True need $mid = left + (right - left + 1) // 2$? Because its True branch does $left = mid$. With the ordinary (lower) midpoint, $left = 3$, $right = 4$ gives $mid = 3$, and $left = mid$ doesn't move $left$ at all — infinite loop. The upper midpoint guarantees $mid > left$ whenever $left < right$, so $left = mid$ always makes progress.

Q: What's the difference between lower bound and "first occurrence"? Lower bound ($\text{first } i \text{ with } \text{nums}[i] \geq \text{target}$) is strictly more general — it returns a meaningful insertion index even when the target doesn't exist in the array at all. First occurrence is just lower bound *when the target happens to be present*.

Q: A candidate says "binary search only works on sorted arrays." How do you correct that? Binary search only requires a **monotonic property over some search space** — the search space doesn't have to be an array, and the property doesn't have to be numeric sortedness. Peak finding (Part 8) works on an unsorted array using local slope; binary search on the answer (Parts 3–5) has no array at all, just a monotonic feasibility function; rotated arrays (Part 7) aren't globally sorted but still have a locally sorted half at every midpoint to exploit.

Q: What's the complexity of Koko Eating Bananas, and why isn't it $O(\log n)$? $O(n \log(\max(\text{piles})))$. The binary search itself is $O(\log(\text{range}))$ iterations, but each iteration pays for an $O(n)$ feasibility check (summing hours across all piles). Always account for the cost of the feasibility function separately from the binary search wrapping it.

Q: How would you explain "binary search on the answer" to someone who's only seen array search? Instead of asking "where is this value in my data," ask "what is the smallest/largest value that satisfies some condition, out of a range of candidate values I could try." If checking whether one candidate value works is efficient, and "works" flips monotonically as the candidate grows, you can binary search that range exactly like you'd binary search an array — `mid` is just a candidate answer instead of a candidate index.

Q: Why can't you binary search an unsorted array for "do two elements sum to K"? There's no ordering to exploit — inspecting one element tells you nothing provable about a whole region of the rest of the array. That's the tell for "use a hash map instead" (Section 12.2).

Q: In the rotated-array search, why do you compare against `nums[left]` and `nums[right]` instead of just checking if the array "looks sorted"? Because that comparison is exactly what *proves* which half is sorted: `nums[left] <= nums[mid]` is a rigorous test (not a heuristic) that the left half has no rotation point inside it. Anything less precise risks searching the wrong half.

Q: What's the single biggest tell that a maximum-feasible problem (Last True) is being confused with a minimum-feasible one (First True)? Read the problem statement's superlative literally. "Minimum X such that condition holds" → the predicate is `False...False True...True`, and larger X keeps working, so search for the *first* True. "Maximum X such that condition holds" → the predicate is `True...True False...False`, and larger X eventually breaks, so search for the *last* True. Getting the midpoint bias wrong (Section 2.5) is usually a symptom of mixing these two up.